

UNIVERSIDAD NACIONAL DE COLOMBIA
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial

Problemas Generales de Conteo

Matemáticas Discretas I

Diego Alejandro Robayo Córdoba

Docente: Jhoan Sebastián Tenjo García

Bogotá, Colombia
2 de junio de 2026

1. Introducción

En este documentose describe el funcionamiento y las características algoritmicas de dos problemas seleccionados de conteo: La combinación de contraseñas bajo restricciones y el conteo de caminos minimos para trasladarse en una grilla de un punto a otro con restricciones. La solución de ambos problemas se basa en las propiedades de la combinatoria y técnicas de conteo. Las soluciones se implementan mediante algoritmos que aprovechan las propiedades matemáticas como la permutación, la combinación y el principio de inclusión-exclusión para obtener resultados de manera eficiente.

2. Problema 4: Sistema de conteo de contraseñas

Este módulo permite calcular la cantidad de contraseñas que pueden formarse bajo distintas restricciones configurables por el usuario.

La interfaz se divide en tres secciones principales:

1. **Dimensiones del problema.** En esta sección se especifican la longitud de la contraseña k y el tamaño total del alfabeto n . Además, se puede indicar si se permite o no la repetición de caracteres marcando el checkbox.
2. **Subconjuntos de caracteres.** Aquí se definen los distintos grupos de caracteres que componen el alfabeto. Por defecto se incluyen los grupos de dígitos, letras mayúsculas y símbolos especiales. El usuario puede modificar el tamaño de estos grupos, agregar nuevos subconjuntos o eliminar los existentes según las necesidades del problema.
3. **Restricciones de inclusión.** En esta sección se seleccionan los subconjuntos que deben estar representados en la contraseña. Cada restricción exige la presencia de al menos un carácter perteneciente al subconjunto correspondiente.

1. Dimensiones del Problema

Longitud de la cadena (k):

5

Tamaño del alfabeto (n):

46

☒ Permitir repetición de elementos

Figura 1: Sección de dimensiones del problema.

2. Subconjuntos de Caracteres

Nota de modelado: Verifica siempre los resultados y la lógica de suma de los subconjuntos con el tamaño del alfabeto.

Dígitos	10	x
Mayúsculas	26	x
Símbolos	10	x

+ Agregar subconjunto

Figura 2: Configuración de subconjuntos de caracteres.

3. Restricciones de Inclusión ("Al menos uno")

☒ Debe contener al menos un Dígitos ☒ Debe contener al menos un Mayúsculas ☒ Debe contener al menos un Símbolos

Calcular

Figura 3: Configuración de restricciones de inclusión.

Una vez completada la configuración, el sistema calcula la cantidad total de contraseñas que satisfacen las restricciones establecidas. Para ello se utilizan principios de conteo combinatorio y, cuando existen restricciones de inclusión, el Principio de Inclusión-Exclusión.

El programa también muestra el desarrollo matemático empleado para obtener el resultado, permitiendo visualizar las expresiones utilizadas en el cálculo.

Desarrollo Analítico

(Verificar resultado del procedimiento. Puede que según la complejidad, la cadena aritmética sea incorrecta)

Aplicando conteo estructurado para n elementos y longitud k , obtenemos el siguiente desarrollo:

$$P(46, 4) = \frac{46!}{42!} - \left[\frac{36!}{32!} + \frac{20!}{16!} - \frac{10!}{6!} + \frac{36!}{32!} - \frac{26!}{22!} - \frac{10!}{6!} + 0 \right] = 1341600$$

Figura 4: Resultado obtenido y desarrollo matemático asociado.

2.1. Conteo básico con repetición

Cuando se permite la repetición de caracteres, el número total de contraseñas posibles de longitud k formadas a partir de un alfabeto de tamaño n se calcula utilizando la fórmula:

$$n^k$$

Por ejemplo, si el alfabeto tiene 26 letras y se permiten contraseñas de longitud 5, el número total de contraseñas posibles sería:

$$26^5 = 11881376$$

La complejidad de este cálculo es $O(1)$, ya que se trata de una operación matemática directa.

2.2. Conteo sin repetición

Cuando no se permite la repetición de caracteres, el número total de contraseñas posibles se calcula utilizando la fórmula de permutaciones:

$$P(n, k) = \frac{n!}{(n - k)!}$$

Los factoriales se calculan mediante ciclos de longitud k , de tal manera que la complejidad de este cálculo es $O(k)$.

$$n(n - 1)(n - 2) \dots (n - k + 1)$$

2.3. Restricciones de inclusión

El usuario puede indicar que ciertos subconjuntos del alfabeto deben estar representados al menos una vez en la contraseña. Para resolver este tipo de problemas se primero se cuenta todas las cadenas posibles y restar aquellas que incumplen la condición, es decir, se utiliza el complemento.

Por ejemplo, si la longitud de la contraseña es 4 y sobre el alfabeto hay 26 letras, ¿cuántas cadenas hay si debe tener al menos una vocal?, permitiendo la repetición.

$$\text{Total de cadenas posibles} = 26^4$$

$$\text{Cadenas sin vocales} = 21^4$$

$$26^4 - 21^4 = 456976 - 194481 = 262495$$

2.4. Principio de Inclusión-Exclusión

Cuando existen múltiples restricciones simultaneas, se utiliza el Principio de Inclusión-Exclusión para evitar contar varias veces las contraseñas que incumplen más de una restricción. Por ejemplo, para tres subconjuntos finitos A , B y C , el principio se expresa de la siguiente manera:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

2.5. Generación de subconjuntos requeridos

El programa permite definir múltiples subconjuntos de caracteres y marcar cuales de ellos deben aparecer al menos una vez. Para realizar el conteo, primero se construye una lista que contiene únicamente los subconjuntos marcados como requeridos. Sea r el número de subconjuntos requeridos. El algoritmo genera todas las posibles combinaciones de estos subconjuntos utilizando máscaras binarias, donde cada máscara representa un subconjunto distinto de restricciones incumplidas. Por ejemplo, si existen 3 subconjuntos requeridos A , B y C , las máscaras permiten representar los conjuntos.

$$\{A\}, \{B\}, \{C\}, \{A, B\}, \{A, C\}, \{B, C\}, \{A, B, C\}$$

En total se generan $2^r - 1$ combinaciones, lo que implica una complejidad de $O(2^r)$ para esta parte del algoritmo. Esta enumeración es esencial para aplicar el Principio de Inclusión-Exclusión correctamente y evitar contar varias veces las contraseñas que incumplen más de una restricción.

Esta es la operación más costosa del algoritmo. Para cada uno de los grupos obligatorios se debe recorrer nuevamente los r grupos para determinar cuales están presentes en la máscara binaria y calcular el tamaño total eliminado. Esto viene dado por:

$$O(r \cdot 2^r)$$

2.6. Aplicación del principio de Inclusión-Exclusión

Una vez obtenidos todos los subconjuntos de restricciones, el algoritmo calcula cuántas contraseñas serían posibles si se eliminaran los caracteres pertenecientes a los grupos seleccionados en cada combinación. Por ende, para cada subconjunto:

1. Se suma el tamaño de los grupos eliminados.
2. Se calcula el número de caracteres restantes en el alfabeto.
3. Se cuentan cuantas contraseñas se pueden formar utilizando unicamente esos caracteres.
4. El resultado se suma o resta al conteo total dependiendo de la cantidad de subconjuntos involucrados.

Si el número de grupos seleccionados es impar, el termino se suma al conteo total, mientras que si es par, se resta.

$$|A_1| + |A_2| + |A_3| + \dots$$

$$-|A_1 \cap A_2| - |A_1 \cap A_3| \dots$$

De esta manera se corrige el problema de contar varias veces las contraseñas que incumplen más de una restricción simultaneamente.

2.7. Uso de BigInteger

Se utiliza la clase BigInteger de .NET para manejar números enteros de tamaño arbitrario, lo cual es esencial dado que el número de contraseñas posibles puede crecer exponencialmente con la longitud de la contraseña y el tamaño del alfabeto. BigInteger permite realizar operaciones aritméticas sin preocuparse por el desbordamiento, garantizando que el programa pueda manejar incluso los casos más extremos sin errores.

2.8. Ejemplos

3. Problema 8: Caminos mínimos en una grilla

Este módulo cuenta los caminos mínimos para trasladarse en una grilla rectangular desde (0,0) hasta (a,b), moviéndose únicamente hacia la derecha o hacia arriba. Además del caso base, se permiten agregar obstáculos en la grilla y asignar puntos obligatorios que deben ser visitados durante el recorrido. La interfaz se divide en cuatro secciones principales:

1. **Dimensiones de la grilla.** En esta sección se especifican los valores de a y b que definen el tamaño de la grilla.
2. **Resultado base.** Se muestran los caminos mínimos en su caso base
3. **Grilla.** Se presenta graficamente la grilla con la posibilidad de agregar obstáculos y puntos obligatorios.
4. **Resultado con restricciones.** Se muestra el resultado obtenido al aplicar las restricciones de obstáculos y puntos obligatorios. Se dividen en dos secciones, una para cada tipo de restricción, y se muestra el desarrollo matemático asociado a cada caso.

3.1. Caso base

Para el caso base, sin obstáculos ni puntos obligatorios, el número de caminos mínimos desde (0,0) hasta (a,b) se requieren a movimientos hacia la derecha y b movimientos hacia arriba, lo que se calcula utilizando la fórmula de combinaciones:

$$\binom{a+b}{a} = \binom{a+b}{b}$$

El calculo binomial se realiza mediante operaciones aritmeticas iterativas, con complejidad de

$$O(n)$$

donde n corresponde al menor de los parametros a y b del coeficiente binomial.

3.2. Caminos con puntos obligatorios

Si un camino debe atravesar un punto intermedio P , entonces el problema se divide en dos tramos independientes: desde (0,0) hasta P y desde P hasta (a,b). El número total de caminos que pasan por P se obtiene con el principio de multiplicación, multiplicando el número de caminos de cada tramo. Si N es el número de caminos totales, entonces

$$N = N_{origen,P} \cdot N_{P,destino}$$

Cuando existen más de un punto obligatorio, estos se ordenan según su posición en la grilla y se aplica el mismo principio de multiplicación para cada tramo entre puntos consecutivos. En cuanto a su complejidad, al incorporar estos puntos obligatorios, el espacio combinatorio se descompone multiplicativamente en subgrillas independientes. Computacionalmente, esto fragmenta el bucle del binomial en segmentos más pequeños, lo que puede reducir significativamente el tiempo de cálculo, manteniendo la complejidad general en $O(n)$, pero con constantes mucho menores debido a la división del problema.

3.3. Caminos con puntos bloqueados

Para los caminos con puntos bloqueados se utiliza el principio de complemento. Primero se calculan los caminos totales y posteriormente se restan aquellos que atraviesan cada punto prohibido. Para un punto bloqueado B .

$$N_B = N_{origen,B} \cdot N_{B,destino}$$

Si existen múltiples puntos bloqueados, se debe aplicar el principio de inclusión-exclusión para evitar contar varias veces los caminos que atraviesan más de un punto bloqueado simultáneamente. Si llegaran a existir, varios puntos bloqueados, calcular los caminos mediante las formulas combinatorias llega a ser inviable debido a que las intersecciones entre los puntos bloqueados pueden ser complejas de determinar. Para solucionarlo se implementa un algoritmo de programación dinámica.

3.4. Programación dinámica en la grilla

Como las restricciones pueden ser utilizadas simultáneamente, usa la programación dinámica para poder realizar los calculos necesarios y obtener el camino óptimo. Se utiliza una matriz dp , donde cada posición almacena la cantidad de caminos válidos para llegar a ese punto. Ya que cualquier camino que llegue al punto (x,y) debe provenir necesariamente desde la izquierda o abajo, la relación de recurrencia utilizada es:

$$dp(x, y) = dp(x - 1, y) + dp(x, y - 1)$$

- **Caminos bloqueados.** Si una coordenada (x,y) corresponde a un punto bloqueado, entonces se asigna $dp(x, y) = 0$, lo que indica que no hay caminos válidos que pasen por ese punto. Esto se traduce en que, como la celda aporta cero caminos, las celdas que dependen de ella (la de la derecha y la de arriba) también se verán afectadas, bloqueando efectivamente el flujo de rutas a través de ese punto.
- **Puntos obligatorios.** Como se mencionó anteriormente, para los puntos obligatorios, se descomponene la trayectoria en una secuencia ordenada de subgrillas independientes.

$$\text{Secuencia} = [(0, 0) \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow (a, b)]$$

Para mantener la validez del orden del tramo, se aplica la condición de que cada punto obligatorio debe ser alcanzable desde el punto anterior en la secuencia, es decir, un punto posterior no puede estar a la izquierda ni abajo de un punto anterior. Luego, para cada tramo se calcula su dp correspondiente y se multiplican los resultados (Princio multiplicativo). Si el resultado de la dp en un solo segmento es cero, entonces el resultado total también será cero, lo que indica que no existe ningún camino válido que cumpla con todas las restricciones simultáneamente.

El algoritmo de programación dinámica tiene una complejidad de $O(a \cdot b)$, ya que se debe llenar una matriz de tamaño a por b . A pesar de que puede aumentar significativamente según las restricciones, sigue siendo eficiente para tamaños moderados de grilla y se mantiene dentro de la misma complejidad.

3.5. Ejemplos

Cinco ejemplos.

4. Visualización matemática mediante MathJax

Con el fin de mostrar las expresiones combinatorias de forma legible, la aplicación utiliza la biblioteca MathJax. Esta herramienta permite renderizar expresiones escritas en sintaxis LaTeX directamente dentro de la interfaz web desarrollada con Blazor.

Después de cada cálculo, el sistema genera dinámicamente la expresión matemática correspondiente al problema resuelto. Dependiendo de la configuración seleccionada por el usuario, la fórmula puede representar:

- Potencias para el conteo con repetición.
- Permutaciones para el conteo sin repetición.
- Expresiones derivadas del Principio de Inclusión-Exclusión.
- Coeficientes binomiales para el conteo de caminos en grillas.

Las expresiones son construidas en tiempo de ejecución y enviadas a MathJax mediante interoperabilidad JavaScript desde Blazor. Esto permite mostrar tanto el procedimiento matemático como el resultado numérico obtenido por el algoritmo.